

claude-windows / da34752f-c664-4262-9602-f5d4cfcc6db4

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\da34752f-c664-4262-9602-f5d4cfcc6db4\subagents\workflows\wf_c5f5bca5-d2a\agent-a3386c2fd2beaeb5e.jsonl`
- SHA-256: `a939c214510c826ded481cc5c66eb2a4a22eff8ab0b96c1b14f7c729ea655cbf`
- Source modified: `2026-06-12T10:46:03+00:00`
- Imported at: `2026-07-05T16:48:22+00:00`
- project: `wf_c5f5bca5-d2a`
- session_id: `da34752f-c664-4262-9602-f5d4cfcc6db4`
```

Transcript

1. user (2026-06-12T10:43:40.319Z)

Repo: C:\Users\avidu\Projects\sports-data-collector (Windows). Review commit fd530b5 on branch feat/prep-annotation-proxy (diff vs master: run `git diff master...feat/prep-annotation-proxy` in that repo, or read the files directly).

Changed files:

- src/core/proxy.py (NEW - probe/encode/verify/hash/manifest engine)
- src/cli/curate.py (new prep_annotation_proxy method + argparse subparser + dispatch)
- tests/test_proxy.py (NEW - 22 offline tests)
- docs/INGESTION_PIPELINE.md (step 0 + 2 FAQ entries)

Purpose (ADR-002 in docs/DECISIONS.md): re-encode a source video into a downscaled "annotation proxy" for an annotation vendor, while preserving the EXACT frame count / fps / timeline so frame N on the proxy == frame N on the original (the vendor's CVAT labels are recomputed as seconds=frame/fps by src/parsers/cvat/rally_cvat.py). Invariants: never change fps, never trim, verify parity after encode (delete proxy on failure), record provenance (source+proxy MD5) in a manifest CSV. Suite is 159 green; a live ffmpeg-8.1.1 E2E smoke passed.

A reviewer claims the following defect. ADVERSARIALLY VERIFY it by reading the actual code/files ? try to REFUTE it. Default to refuted if the scenario cannot actually occur or the behavior is acceptable/intended for this tool's scope.

CLAIM [low] Odd source heights below the cap produce an odd scale output height ? libx264 + yuv420p fails the encode (C:\Users\avidu\Projects\sports-data-collector\src\core\proxy.py:171) `scale=-2:min(1080\,ih)` forces the WIDTH even (-2) but passes the height through: for any source with ih < height and ih odd (cropped exports, screen captures like 1438x807), the output height is odd and libx264 with yuv420p aborts ('height not divisible by 2'). Loud failure, not silent corruption ? but combined with the missing encode-failure cleanup it also strands a partial proxy. The `\\` escaping itself is correct for a no-shell list-arg on Windows (no quotes/spaces, so subprocess passes the backslash verbatim and ffmpeg unescapes the comma at the filtergraph level) ? only the evenness is missing. Proposed fix: Use scale=-2:2*trunc(min(1080\,ih)/2) (keeping the comma escape) so the height is always even.

2. user (2026-06-12T10:43:45.933Z)

fd530b5 feat(curate): prep-annotation-proxy - frame-identical vendor handoff re-encode (ADR-002) 4fb2d6f docs: ADR-002 - vendor annotation proxies generated in-repo (curate prep-annotation-proxy) (#21)

a2d08f8 docs(cvat): update detect_issues padding rationale to config-driven values (#20)

```
docs/INGESTION_PIPELINE.md | 29 ++++
src/cli/curate.py           | 147 +++++
src/core/proxy.py           | 242 +++++
tests/test_proxy.py         | 309 +++++
4 files changed, 724 insertions(+), 3 deletions(-)
```

3. user (2026-06-12T10:43:46.706Z)

```
1      """Annotation-proxy generation for vendor handoff (ADR-002).
2
3      Vendors annotate in CVAT, where every rally timestamp is later recomputed from
4      *frame indices* (`seconds = frame / fps` ? see `src/parsers/cvat/rally_cvat.py`).
5      A 4K/59.94 source is too heavy to frame-step, which is what pushed the pilot
6      vendor onto a ~30-frame sampling grid (~0.5 s) and broke the  $\pm 0.3$  s boundary bar.
7      The fix is to hand the vendor a light **annotation proxy** of the *same timeline*:
8      downscaled and scrub-friendly, but identical in frame count, fps, and duration,
9      so frame N on the proxy IS frame N on the original and returned labels apply to
10     the original unchanged.
11
12     The ADR-002 proxy contract enforced here:
13     - video timestamps pass through untouched (`-fps_mode passthrough`, never
14       ``-r``): same frame count, same fps, no trim, no leading padding;
15     - the proxy is VERIFIED against the source (frame count / fps / duration) and a
16       failed proxy must be deleted, never shared ? a silently-wrong proxy poisons
17       golden labels;
18     - provenance is recorded (source + proxy MD5s) so the proxy?original mapping
19       survives the handoff (the indexer keys videos by file MD5; downstream
20       processing must keep running on the ORIGINAL file).
21     """
22     import csv
23     import hashlib
24     import os
25     import shutil
26     import subprocess
27     from dataclasses import dataclass
28     from typing import Any, Dict, List, Optional
29
30
31     class ProxyError(RuntimeError):
32         """A proxy step failed in a way that must stop the handoff (never degrade)."""
33
34
35     #: Columns of the provenance manifest (one row appended per generated proxy).
36     MANIFEST_FIELDS = [
37         "video_id", "source_path", "source_md5", "proxy_path", "proxy_md5",
38         "fps", "frame_count", "duration_s", "source_resolution", "proxy_resolution",
39         "encoder_settings", "created_at",
40     ]
41
42     # Encode defaults (see ADR-002). Height 1080 not 720 (table-tennis ball
43     # visibility); GOP 15 = a keyframe every ~0.25 s at 59.94 fps for snappy
44     # back-stepping; CRF 19 keeps the ball/shuttle legible.
45     DEFAULT_HEIGHT = 1080
46     DEFAULT_CRF = 19
47     DEFAULT_GOP = 15
48     DEFAULT_PRESET = "medium"
49
50
51     @dataclass
52     class ProbeInfo:
53         """The frame?time identity of one video stream, as ffprobe reports it."""
54         width: Optional[int] = None
55         height: Optional[int] = None
56         avg_fps: Optional[float] = None      # avg_frame_rate (frames / duration)
57         r_fps: Optional[float] = None        # r_frame_rate (the container's base rate)
58         nb_frames: Optional[int] = None      # demuxed packet count == frame count
59         duration: Optional[float] = None     # video stream duration, else container
60
61     @property
62     def resolution(self) -> str:
63         if self.width and self.height:
```

```

64         return f"{self.width}x{self.height}"
65     return "unknown"
66
67
68 def parse_rate(value: Optional[str]) -> Optional[float]:
69     """Parse an ffmpeg rational rate ('60000/1001', '30/1') to float fps."""
70     if not value:
71         return None
72     try:
73         if "/" in value:
74             num, den = value.split("/", 1)
75             num_f, den_f = float(num), float(den)
76             if den_f == 0:
77                 return None
78             return num_f / den_f
79         return float(value)
80     except ValueError:
81         return None
82
83
84 def probe_video(path: str) -> ProbeInfo:
85     """Probe the frame?time identity of `path` via ffmpeg (loud on failure).
86
87     Uses ``-count_packets`` (demux-only, no decode) so the frame count is exact
88     yet cheap even for a 30-minute file. Unlike the fetch path's best-effort
89     probe, a failure here RAISES: parity verification is the whole point of the
90     proxy flow, so a missing ffmpeg must stop it, not degrade it.
91     """
92     if not shutil.which("ffmpeg"):
93         raise ProxyError("ffmpeg not found on PATH (install FFmpeg)")
94     if not os.path.exists(path):
95         raise ProxyError(f"video not found: {path}")
96     import json as _json
97     try:
98         out = subprocess.run(
99             ["ffmpeg", "-v", "error", "-select_streams", "v:0",
100              "-count_packets",
101              "-show_entries",
102              "stream=width,height,avg_frame_rate,r_frame_rate,nb_read_packets,duration"
103              ":format=duration",
104              "-of", "json", path],
105             stdout=subprocess.PIPE, stderr=subprocess.PIPE, text=True, check=True,
106         ).stdout
107         doc = _json.loads(out)
108     except (subprocess.SubprocessError, OSError, ValueError) as e:
109         raise ProxyError(f"ffmpeg failed on {path}: {e}")
110
111     streams = doc.get("streams") or []
112     if not streams:
113         raise ProxyError(f"no video stream found in {path}")
114     s = streams[0]
115
116     def _f(value) -> Optional[float]:
117         try:
118             return float(value)
119         except (TypeError, ValueError):
120             return None
121
122     def _i(value) -> Optional[int]:
123         try:
124             return int(value)
125         except (TypeError, ValueError):
126             return None
127
128     # Prefer the video stream's own duration; fall back to the container's

```

```

129     # (audio encoder priming can stretch the container a few hundredths).
130     duration = _f(s.get("duration"))
131     if duration is None:
132         duration = _f((doc.get("format") or {}).get("duration"))
133
134     return ProbeInfo(
135         width=_i(s.get("width")),
136         height=_i(s.get("height")),
137         avg_fps=parse_rate(s.get("avg_frame_rate")),
138         r_fps=parse_rate(s.get("r_frame_rate")),
139         nb_frames=_i(s.get("nb_read_packets")),
140         duration=duration,
141     )
142
143
144 def is_vfr(info: ProbeInfo, rel_tol: float = 0.001) -> bool:
145     """True when the stream looks variable-frame-rate (or we cannot tell).
146
147     For CFR camera footage r_frame_rate == avg_frame_rate. A mismatch means the
148     `frame / fps` time recompute is approximate ? conservative callers should
149     require an explicit opt-in before proxying such a source.
150     """
151     if info.avg_fps is None or info.r_fps is None or info.avg_fps == 0:
152         return True
153     return abs(info.r_fps - info.avg_fps) / info.avg_fps > rel_tol
154
155
156 def build_proxy_command(src: str, dst: str, height: int = DEFAULT_HEIGHT,
157                        crf: int = DEFAULT_CRF, gop: int = DEFAULT_GOP,
158                        preset: str = DEFAULT_PRESET) -> List[str]:
159     """The ffmpeg invocation that downscales WITHOUT touching the timeline.
160
161     Invariants (ADR-002): no ``-r`` / fps filter (fps must never change), no
162     trim, ``-fps_mode passthrough`` (ffmpeg >= 5.1) so every source frame maps
163     1:1 onto a proxy frame with its original timestamp. ``min(height, ih)``
164     never upscales. Audio is kept (contact sound is the annotator's best
165     boundary cue).
166     """
167     return [
168         "ffmpeg", "-hide_banner", "-loglevel", "warning", "-stats", "-y",
169         "-i", src,
170         "-map", "0:v:0", "-map", "0:a?",
171         "-vf", f"scale=-2:min({height}\\,ih)",
172         "-c:v", "libx264", "-preset", preset, "-crf", str(crf),
173         "-g", str(gop), "-pix_fmt", "yuv420p",
174         "-fps_mode", "passthrough",
175         "-c:a", "aac", "-b:a", "160k",
176         "-movflags", "+faststart",
177         dst,
178     ]
179
180
181 def run_ffmpeg(cmd: List[str]) -> None:
182     """Run an ffmpeg command, streaming its progress to the console."""
183     if not shutil.which("ffmpeg"):
184         raise ProxyError("ffmpeg not found on PATH (install FFmpeg)")
185     try:
186         # No capture: -stats progress and any warnings go straight to the user.
187         subprocess.run(cmd, check=True)
188     except (subprocess.SubprocessError, OSError) as e:
189         code = getattr(e, "returncode", "?")
190         raise ProxyError(f"ffmpeg failed (exit {code}) ? proxy not produced")
191
192
193 def verify_parity(src: ProbeInfo, proxy: ProbeInfo) -> List[str]:

```

```

194     """Check the proxy carries the source's exact frame?time identity.
195
196     Returns a list of human-readable failures (empty == parity holds). Frame
197     count must match EXACTLY; fps within 0.1%; duration within a small slack
198     (AAC priming can pad the container by a few hundredths of a second) that
199     still catches any real trim.
200     """
201     errors = []
202
203     if src.nb_frames is None or proxy.nb_frames is None:
204         errors.append("frame count unavailable on source and/or proxy ? cannot prove
parity")
205     elif src.nb_frames != proxy.nb_frames:
206         errors.append(f"frame count changed: source {src.nb_frames} -> proxy
{proxy.nb_frames}")
207
208     if src.avg_fps is None or proxy.avg_fps is None:
209         errors.append("fps unavailable on source and/or proxy ? cannot prove parity")
210     elif abs(proxy.avg_fps - src.avg_fps) / src.avg_fps > 0.001:
211         errors.append(f"fps changed: source {src.avg_fps:.3f} -> proxy
{proxy.avg_fps:.3f}")
212
213     if src.duration is None or proxy.duration is None:
214         errors.append("duration unavailable on source and/or proxy ? cannot prove
parity")
215     else:
216         tol = max(2.0 / src.avg_fps, 0.2) if src.avg_fps else 0.2
217         if abs(proxy.duration - src.duration) > tol:
218             errors.append(f"duration changed: source {src.duration:.3f}s -> "
219                           f"proxy {proxy.duration:.3f}s (tolerance {tol:.3f}s)")
220
221     return errors
222
223
224 def md5_file(path: str, chunk_size: int = 1024 * 1024) -> str:
225     """MD5 of a file's bytes (the identity downstream tooling keys videos by)."""
226     digest = hashlib.md5()
227     with open(path, "rb") as f:
228         for chunk in iter(lambda: f.read(chunk_size), b""):
229             digest.update(chunk)
230     return digest.hexdigest()
231
232
233 def append_manifest_row(manifest_path: str, row: Dict[str, Any]) -> None:
234     """Append one provenance row, writing the header only on first creation."""
235     parent = os.path.dirname(os.path.abspath(manifest_path))
236     os.makedirs(parent, exist_ok=True)
237     is_new = not os.path.exists(manifest_path) or os.path.getsize(manifest_path) == 0
238     with open(manifest_path, "a", newline="", encoding="utf-8") as f:
239         writer = csv.DictWriter(f, fieldnames=MANIFEST_FIELDS)
240         if is_new:
241             writer.writeheader()
242         writer.writerow({k: row.get(k, "") for k in MANIFEST_FIELDS})
243

```

4. user (2026-06-12T10:44:12.247Z)

```

1     import os
2     import sys
3     import time
4     import json
5     import csv
6     import argparse
7     from typing import List, Dict, Any, Tuple
8     from tabulate import tabulate

```

```

9
10 from src.core.config import load_config
11 from src.core.parsing import safe_int as _safe_int, safe_float_required as _safe_float
12 from src.core.db import SportsDatabase
13 from src.core.schemas.base import VideoMetadata, SportType, LicenseType, CurationStatus
14 from src.core.schemas.badminton import BadmintonRally
15 from src.core.schemas.tennis import TennisPoint
16 from src.core.schemas.cricket import CricketDelivery
17 from src.core.schemas.pickleball import PickleballRally
18 from src.core.schemas.tabletennis import TableTennisRally
19 from src.core.validator import DatasetValidator
20 from src.core.downloader import download_best_video, probe_resolution_fps,
height_to_label
21
22
23 class SportsDataCLI:
24     def __init__(self, config_path: str = "config/config.yaml"):
25         self.config_path = config_path
26         self.config = self._load_config()
27
28         # Resolve DB path
29         db_path = self.config.get("database_path", "data/sports_metadata.db")
30         self.db = SportsDatabase(db_path)
31         self.validator = DatasetValidator(config_path)
32
33     def _load_config(self) -> Dict[str, Any]:
34         if not os.path.exists(self.config_path):
35             print(f"Warning: Configuration file not found at {self.config_path}. Using
defaults.")
36         return load_config(self.config_path)
37
38     def status(self):
39         """Displays status summary of all videos in the database."""
40         # Query total count by status via the public DB read API.
41         rows = self.db.get_status_counts() # list of (sport, status, cnt)
42
43         if not rows:
44             print("\nDatabase is currently empty. Run crawlers or import local data.\n")
45             return
46
47         data = [[sport, status, cnt] for sport, status, cnt in rows]
48         print("\n=== Dataset Ingestion Status ===")
49         print(tabulate(data, headers=["Sport", "Status", "Count"], tablefmt="grid"))
50         print()
51
52     @staticmethod
53     def _annotation_key(sport_str, a):
54         """The per-sport primary-key value of one annotation (for duplicate
detection)."""
55         if sport_str in ("badminton", "pickleball", "tabletennis"):
56             return getattr(a, "rally_number", None)
57         if sport_str == "tennis":
58             return getattr(a, "point_number", None)
59         if sport_str == "cricket":
60             return (getattr(a, "over", None), getattr(a, "ball", None))
61         return None
62
63     def _guard_import(self, sport_str, video_id, annotations, force=False):
64         """Pre-insert safety checks (run BEFORE any DB write, so a rejection is
atomic)."""
65         # (1) Duplicate primary-key guard. int(float()) truncation (e.g. 7 and 7.5 both
66         # become 7) would otherwise raise IntegrityError mid-insert, leaving a committed
67         # video row with rolled-back annotations. Catch it before touching the DB.
68         keys = [self._annotation_key(sport_str, a) for a in annotations]
69         keys = [k for k in keys if k is not None]

```

```

70         seen, dupes = set(), set()
71     for k in keys:
72         if k in seen:
73             dupes.add(k)
74             seen.add(k)
75     if dupes:
76         print(f"Error: duplicate rally/point identifier(s) {sorted(dupes)} in the
annotations "
77               f"(often a fractional id like 7.5 truncating to 7).")
78         print(" -> re-run with --normalize, or normalize first: "
79               "python -m src.cli.normalize_annotations --in <csv> --out clean.csv")
80         sys.exit(1)
81
82     # (2) Golden-label protection ? refuse to silently clobber an existing video's
83     # annotations (filename-derived video_ids collide easily).
84     if self.db.get_video(video_id):
85         try:
86             n = len(self.db.get_segment_intervals(video_id, SportType(sport_str)))
87         except Exception:
88             n = 0
89         if n > 0 and not force:
90             print(f"Error: video_id '{video_id}' already has {n} annotated
segment(s); "
91                   f"importing would REPLACE them.")
92             print(" -> pass --force to overwrite, or use a distinct --video-id to
keep both.")
93             sys.exit(1)
94         if n > 0:
95             print(f"Warning: --force given ? replacing {n} existing segment(s) for
video_id '{video_id}'.")
96
97     def import_local(self, args):
98         """Manually registers a local video and annotation file into the database."""
99         sport_str = args.sport.lower()
100         if sport_str not in self.config.get("supported_sports", ["badminton"]):
101             print(f"Error: Sport '{sport_str}' is not supported. Supported:
{self.config.get('supported_sports')}")
102             sys.exit(1)
103
104         # Validate video path
105         if not os.path.exists(args.video_path):
106             print(f"Warning: Local video path '{args.video_path}' does not exist on
disk, but continuing registration.")
107
108         # Validate annotation file
109         if not os.path.exists(args.annotations_path):
110             print(f"Error: Annotations file '{args.annotations_path}' not found.")
111             sys.exit(1)
112
113         match_name = args.match_name or os.path.basename(args.video_path).split('.')[0]
114         video_id = args.video_id or match_name.lower().replace(" ", "_")
115
116         # Parse license
117         try:
118             license_type = LicenseType(args.license)
119         except ValueError:
120             print(f"Warning: Unknown license '{args.license}'. Defaulting to Unknown
(non-commercial).")

```

5. user (2026-06-12T10:44:19.015Z)

```

387-         reason = result.error or "unknown error"
388-     else:
389-         reason = (f"got {result.resolution_label} (< min-height {args.min_height}p)
? "

```

```

390-             f"YouTube served a low format (rate-limited?)")
391-         try:
392-             os.remove(result.path)
393-         except OSError:
394-             pass
395-
396-         if i < attempts - 1:
397-             backoff = args.retry_backoff * (i + 1)
398-         --
641-         print(f"Skipped (non-commercial) : {skipped_noncomm} (pass --include-
noncommercial to include)")
642-         if skipped_empty:
643-             print(f"Skipped (no annotations) : {skipped_empty}")
644-         print(f"Manifest           : {manifest_path}\n")
645-
646-     def prep_annotation_proxy(self, args):
647-         """Re-encode a video into a frame-identical annotation proxy (ADR-002).
648-
649-         The proxy is what the annotation vendor receives: downscaled and
650-         scrub-friendly (dense keyframes) so CVAT frame-step-1 work is cheap,
651-         but with the source's EXACT frame count / fps / timeline, so frame N
652-         --
653-
654-         crf=args.crf, gop=args.gop,
655-         preset=args.preset)
656-
657-     print(f"Encoding proxy ({src_info.resolution} -> height<={args.height}, "
658-           f"f"fps {src_info.avg_fps:g} preserved)...")
659-
660-     try:
661-         proxylib.run_ffmpeg(cmd)
662-         proxy_info = proxylib.probe_video(out)
663-     except proxylib.ProxyError as e:
664-         print(f"Error: {e}")
665-         sys.exit(1)
666-
667-     --
668-
669-     if errors:
670-         print("Error: proxy FAILED frame<->time parity verification:")
671-         for err in errors:
672-             print(f" - {err}")
673-
674-         try:
675-             os.remove(out)
676-             print(f"Deleted the failed proxy '{out}' (a silently-wrong proxy "
677-                   f"f"poisons golden labels).")
678-         except OSError:
679-             print(f"WARNING: could not delete the failed proxy '{out}' ? do NOT "
680-                   f"f"share it with the vendor.")
681-
682-     --
1221-     elif args.command == "curate":
1222-         cli.curate()
1223-     elif args.command == "import-local":
1224-         cli.import_local(args)
1225-     elif args.command == "prep-annotation-proxy":
1226-         cli.prep_annotation_proxy(args)
1227-     elif args.command == "convert-cvat":
1228-         cli.convert_cvat(args)
1229-     elif args.command == "validate-delivery":
1230-         cli.validate_delivery(args)
1231-     elif args.command == "export":

```

6. user (2026-06-12T10:44:47.151Z)

```

646     def prep_annotation_proxy(self, args):
647         """Re-encode a video into a frame-identical annotation proxy (ADR-002).
648
649         The proxy is what the annotation vendor receives: downscaled and
650         scrub-friendly (dense keyframes) so CVAT frame-step-1 work is cheap,

```



```

651 but with the source's EXACT frame count / fps / timeline, so frame N
652 on the proxy is frame N on the original and returned labels apply to
653 the original unchanged. Parity is verified after the encode; a proxy
654 that fails verification is deleted, never shared. Provenance (source +
655 proxy MD5s) is appended to the proxy manifest ? downstream processing
656 (import-local, export, the indexer) keeps running on the ORIGINAL file.
657 """
658 from src.core import proxy as proxylib
659
660 src = args.video_path
661 if not os.path.exists(src):
662     print(f"Error: video '{src}' not found.")
663     sys.exit(1)
664
665 stem = os.path.basename(src).split('.')[0]
666 # Mirror import-local's video_id derivation so the manifest row and a
667 # later DB registration of the same source agree on the key.
668 video_id = args.video_id or stem.lower().replace(" ", "_")
669 out = args.out or os.path.join("data", "videos", "proxies", f"{stem}_proxy.mp4")
670 if os.path.abspath(out) == os.path.abspath(src):
671     print("Error: --out must differ from --video-path.")
672     sys.exit(1)
673 if os.path.exists(out) and not args.force:
674     print(f"Error: proxy '{out}' already exists; pass --force to overwrite.")
675     sys.exit(1)
676
677 try:
678     src_info = proxylib.probe_video(src)
679 except proxylib.ProxyError as e:
680     print(f"Error: {e}")
681     sys.exit(1)
682 if src_info.avg_fps is None:
683     print(f"Error: could not determine fps of '{src}' ? the frame<->time "
684           f"contract needs it.")
685     sys.exit(1)
686 if proxylib.is_vfr(src_info) and not args.allow_vfr:
687     print(f"Error: '{src}' looks variable-frame-rate "
688           f"(r={src_info.r_fps} vs avg={src_info.avg_fps}).")
689     print(" The vendor pipeline recomputes times as frame/fps, which is only "
690           "exact for constant-frame-rate sources. Re-run with --allow-vfr to "
691           "proceed anyway (timestamps stay identical to the original, but "
692           "frame/fps seconds are approximate for BOTH files).")
693     sys.exit(1)
694
695 os.makedirs(os.path.dirname(os.path.abspath(out)), exist_ok=True)
696 cmd = proxylib.build_proxy_command(src, out, height=args.height,
697                                   crf=args.crf, gop=args.gop,
698                                   preset=args.preset)
699 print(f"Encoding proxy ({src_info.resolution} -> height<={args.height}, "
700       f"f"fps {src_info.avg_fps:g} preserved)...")
701 try:
702     proxylib.run_ffmpeg(cmd)
703     proxy_info = proxylib.probe_video(out)
704 except proxylib.ProxyError as e:
705     print(f"Error: {e}")
706     sys.exit(1)
707
708 errors = proxylib.verify_parity(src_info, proxy_info)
709 if errors:
710     print("Error: proxy FAILED frame<->time parity verification:")
711     for err in errors:
712         print(f" - {err}")
713     try:
714         os.remove(out)
715     print(f"Deleted the failed proxy '{out}' (a silently-wrong proxy ")

```

```

716         f"poisons golden labels).")
717     except OSError:
718         print(f"WARNING: could not delete the failed proxy '{out}' ? do NOT "
719               f"share it with the vendor.")
720     sys.exit(1)
721
722     print("Hashing source + proxy (provenance)...")
723     source_md5 = proxylib.md5_file(src)
724     proxy_md5 = proxylib.md5_file(out)
725
726     from datetime import datetime, timezone
727     proxylib.append_manifest_row(args.manifest, {
728         "video_id": video_id,
729         "source_path": os.path.abspath(src),
730         "source_md5": source_md5,
731         "proxy_path": os.path.abspath(out),
732         "proxy_md5": proxy_md5,
733         "fps": f"{src_info.avg_fps:g}",
734         "frame_count": src_info.nb_frames,
735         "duration_s": f"{src_info.duration:.3f}" if src_info.duration is not None
736     },
737     else "",
738         "source_resolution": src_info.resolution,
739         "proxy_resolution": proxy_info.resolution,
740         "encoder_settings": (f"libx264 crf={args.crf} g={args.gop} "
741                             f"preset={args.preset} height<={args.height} "
742                             f"fps_mode=passthrough audio=aac160k"),
743         "created_at": datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ"),
744     })
745
746     src_mb = os.path.getsize(src) / (1024 * 1024)
747     out_mb = os.path.getsize(out) / (1024 * 1024)
748     print("\n=== Annotation proxy ready ===")
749     print(f"  Video ID      : {video_id}")
750     print(f"  Source          : {src} ({src_info.resolution}, {src_mb:.0f} MB, md5 {source_md5})")
751     print(f"  Proxy           : {out} ({proxy_info.resolution}, {out_mb:.0f} MB, md5 {proxy_md5})")
752     print(f"  Parity          : VERIFIED ? {src_info.nb_frames} frames @ {src_info.avg_fps:g} fps on both")
753     print(f"  Manifest        : {args.manifest}")
754     print("\nNext:")
755     print("  1. Share the PROXY with the vendor (CVAT at frame step 1; first/last "
756         "box of every rally on the exact contact/dead frame).")
757     print("  2. On delivery, convert-cvat may use --video-path with either file "

```

7. user (2026-06-12T10:45:16.815Z)

```
'scale=-2:min(1080\\,ih)'
```

8. user (2026-06-12T10:45:17.941Z)

```

37:     assert cmd[cmd.index("-vf") + 1] == "scale=-2:min(1080\\,ih)"
46:     assert cmd[cmd.index("-vf") + 1] == "scale=-2:min(720\\,ih)"

```

9. user (2026-06-12T10:45:55.790Z)

Structured output provided successfully

10. assistant (2026-06-12T10:46:03.270Z)

Verification complete ? the claim is confirmed as a real, low-severity defect.